MÉMOIRE OSINT

Livrable

Pipeline de données pour l'IA

Collecte TASS et corpus historique

Repère	Version présentée
Auteur	Edouard Cappaert
Code	Code_OSINT_Bloc3.zip et dépôt GitHub memoire-osint
Date	18/10/2026
Statut	Application locale testée, collecte périodique et essais fictifs identifiés

Les preuves distinguent corpus historique, collecte réelle et fixtures synthétiques. La soutenance personnelle et la validation du jury restent à observer.

Sommaire

Introduction et analyse des données 3

Schéma du pipeline et stack technique 4

Code principal et reprise automatique 5

Problèmes rencontrés, droits et sécurité 6

Analyse des données et résultats mesurés 7

Captures d’exécution et monitoring 8

Documentation d’utilisation 9

Conclusion, soutenance et sources 10

Lire les états de preuve

Prévu : préparé sans exécution observée. Implémenté : présent dans le code. Testé : scénario exécuté et rattaché à une version. Démontré au jury dépend de la prestation réelle. Les 33 critères exacts de Bloc 3!A6:A38 sont reliés à leur preuve dans Correspondance_criteres_Bloc3.json.

La chaîne locale automatise découverte, téléchargement, qualité et publication. Le mode périodique relance des micro-lots sur une fenêtre glissante sans nouveau formulaire. La collecte manuelle reste disponible. L’import dans le Bloc 2 et l’inférence du Bloc 4 restent des opérations distinctes.

Introduction et analyse des données

Contexte et problématique

La cellule fictive de veille documentaire recherche des mentions WEAPON, MIL_UNIT et MIL_ORG. Le pipeline prépare des textes stables pour l'architecture Bloc 2 et l'inférence Bloc 4. Il doit tolérer les erreurs isolables, préserver la dernière publication valide et reprendre sans perte ni doublon.

Deux sources identifiées

Le corpus historique provient du JSON fourni : 21 742 entrées brutes, 66 textes vides rejetés et 21 676 textes nettoyés. L'extension découvre les URLs sur les rubriques publiques TASS anglais, puis extrait le corps et les métadonnées des pages d'articles. Le catalogue observé avant finalisation compte 21 681 articles après cinq nouveaux articles réels.

La pagination /userApi/categoryNewsList est le point d'entrée utilisé par les pages du site ; ce n'est pas une API publique contractuelle. Les bornes portent sur la date de publication UTC, avec début et fin inclusifs. La couverture dépend des archives accessibles, des pages et de la limite d'articles.

Champ	Contrat
id, date, URL	Identifiant conservé, epoch et date lisible UTC, URL HTTPS TASS.
title, text	Titre et corps éditorial ; pas d'images, profils ou coordonnées collectés séparément.
text_sha256	Empreinte du texte UTF-8 exact utilisé pour les offsets.
provenance, version	Source, exécution, normalisation, date de collecte et empreinte de révision.
offset_unit	unicode_codepoint ; intervalle start inclus, end exclu.

Schéma du pipeline et stack technique



Séparation des responsabilités

Python 3.12 et sa bibliothèque standard : source pour accès et extraction, store pour persistance, worker pour checkpoints et publication, service pour interface et supervision, rights/security/resilience pour les contrôles transversaux. SQLite est un catalogue local ; PostgreSQL et MongoDB restent les stockages métier Bloc 2.

Le serveur écoute sur 127.0.0.1, vérifie l'hôte et protège les commandes par session CSRF. Les textes et exports gérés sont destinés au volume chiffré. Une copie SQLite cohérente et vérifiée maintient la lecture en cas de panne du catalogue principal ; les écritures sont alors bloquées.

Collecte périodique

Le module continuous orchestre des cycles successifs sur les deux derniers jours UTC. La configuration attend 60 secondes après la fin d'un cycle. Le délai total comprend aussi découverte, traitement, publication et reprises. Ce fonctionnement par micro-lots ne garantit pas un temps réel strict.

Code principal et reprise automatique

Checkpoints persistants

La découverte enregistre URLs et curseur ; chaque article téléchargé possède un statut durable. Une interruption permet de reprendre les seules unités restantes. Le superviseur relance automatiquement un worker détruit, avec trois tentatives au maximum. L’arrêt volontaire reste une pause manuelle.

```
with database(state) as db, db:
    db.execute("UPDATE tasks SET status=?,payload=? "
               "WHERE job_id=? AND url=?",
               (status, normalized_payload, key, url))
# worker.py : aucun déplacement du checkpoint avant persistance.
```

Calendrier et reprise

Une transaction crée la tâche et son lien au calendrier. Après arrêt brutal, la reprise conserve la tâche en attente. Une pause volontaire reste prioritaire. Les relances de cycle sont bornées, en plus des tentatives internes du worker. Un lot bloqué par la qualité demande une correction avant reprise.

Qualité avant activation

Le taux de rejet maximal du collecteur est configurable, 20 % par défaut. Il s’agit d’un choix du projet. Si le contrôle bloque un lot, la dernière publication valide est conservée. Les formats JSON et JSONL proviennent du même instantané transactionnel. Le seuil historique de 0,5 % concerne l’import de fichiers.

Scénario	Preuve actuelle
Erreur HTTP	Nouvelles tentatives bornées, rejet isolé et alerte conservée.
Worker détruit	Processus relancé automatiquement ; checkpoints réutilisés sans doublon.
Recollecte	Articles inchangés évités, révisions distinctes conservées.
Droits	Correction persistante et suppression prioritaires sur la recollecte.

Preuves : Tests_collecte_final.log, Tests_cloture_final.log et Frontend_tests.log. Les fixtures servent aux pannes et aux droits ; elles ne constituent pas des faits journalistiques.

Problèmes rencontrés, droits et sécurité

Point	Réponse vérifiée ou limite
Failed to fetch persistant	Le bandeau disparaît après reconnexion ; aucune commande incertaine n’est rejouée automatiquement.
Rectification	Texte normalisé, nouvelle empreinte, anciennes versions incorrectes purgées et décision prioritaire sur les futurs imports.
Suppression	Catalogue, tâches, versions, exports gérés et secours purgés ; checkpoint WAL et compactage SQLite.
Stockage indisponible	Consultation et export du dernier catalogue validé ; collecte et import B2 bloqués.
Incidents	Signal de permissions et d’intégrité, confinement et dossier d’exercice fictif relié à I01.

Protection au repos

Le lanceur exige le volume macOS chiffré AES-256 et ne crée aucun état en clair si le volume est fermé. État observé : volume activé, migration et cycle arrêt, démontage, déverrouillage personnel et relance vérifiés. Les 21 701 articles, tables et exports sont conservés par empreinte. Démarrage strict refusé lorsque le volume est fermé (Cycle_volume_chiffre.json).

Traçabilité et coordination

Consultations, exports et droits enregistrent compte système, date, résultat et références opaques, sans texte, secret ou mots-clés libres. Le compte du serveur ne distingue pas des personnes partageant une session. Les journaux gérés sont purgés au-delà de six mois calendaires ; les décisions actives de droits sont séparées.

La fixture Droits_B3_B2.json vérifie correction, purge et refus de réimport dans SQL/Mongo/index. Les annotations, sources historiques et copies déjà téléchargées ailleurs demandent une coordination. Aucun effacement physique garanti d’un SSD ni validation juridique n’est revendiqué.

Analyse des données et résultats mesurés

Catalogue initial	Articles ajoutés	Médiane	Articles/s	Minimum / maximum
Vide	100	0.78 s	127.5	0.69 / 1.39 s
Vide	1000	12.38 s	80.8	12.29 / 12.76 s
Vide	2000	24.92 s	80.3	23.96 / 26.05 s
21 701 articles	100	2.89 s	34.5	2.82 / 2.95 s
21 701 articles	1000	14.29 s	70.0	14.12 / 14.98 s
21 701 articles	2000	26.17 s	76.4	26.07 / 30.26 s

18 exécutions sur le même volume APFS AES-256, trois répétitions par taille et catalogue. Copies isolées du catalogue rempli, corps HTML fictifs d'environ 1, 10 et 50 Kio. Préparation des copies exclue ; découverte, extraction, checkpoints, exports et copie de secours inclus. Comptes, empreintes et égalité JSON/JSONL vérifiés. Détail, dispersion et volumes : Benchmark_chiffre.json.

Arrivées successives

Un essai complémentaire traite 10 000 articles fictifs d'environ 2 Kio sur 40 cycles en 35,48 s, soit 281,82 articles/s. Les 62 répétitions entre cycles ne créent pas de révision superflue. Délais disponibilité-publication : 0,38 à 1,36 s. Source injectée, sans réseau TASS ni garantie de temps réel strict. Preuve : Complement_20261018/Charge_continue.json.

Interprétation

La publication recopie le catalogue entier vers le secours : son coût fixe pénalise les petits lots. Les durées et débits sont à comparer entre tailles et catalogues ; aucun seuil d'absence de ralentissement significatif n'est fixé par la grille. Poste partagé, caches non neutralisés, collecte bornée à 2 000 candidats. Sans réseau ni mesure isolée du coût du chiffrement. Les anciens benchmarks restent historiques.

Qualité et archives

Le corpus historique est identifié par empreinte ; les textes normalisés conservent les IDs, dates et offsets. Les vérifications réseau sont bornées et séparées dans Reseau_final.json. Une période vide ou une limite atteinte ne prouve pas l'absence d'autres publications. Les mentions extraites reflètent le corpus éditorial, pas un inventaire militaire réel.

Les tests de restauration utilisent une ancienne copie fictive et vérifient l'application des droits avant retour à la lecture. La continuité démontrée porte sur la lecture locale ; la collecte ne continue pas en cas de défaillance du stockage principal.

Captures d’exécution et monitoring

Le suivi conserve l’interface du collecteur et affiche maintenant les cycles, la prochaine échéance et la durée du dernier cycle. La capture ci-dessous provient de dix cycles sur une source fictive. Les nouvelles vérifications restent distinctes des mesures réseau et des captures antérieures.

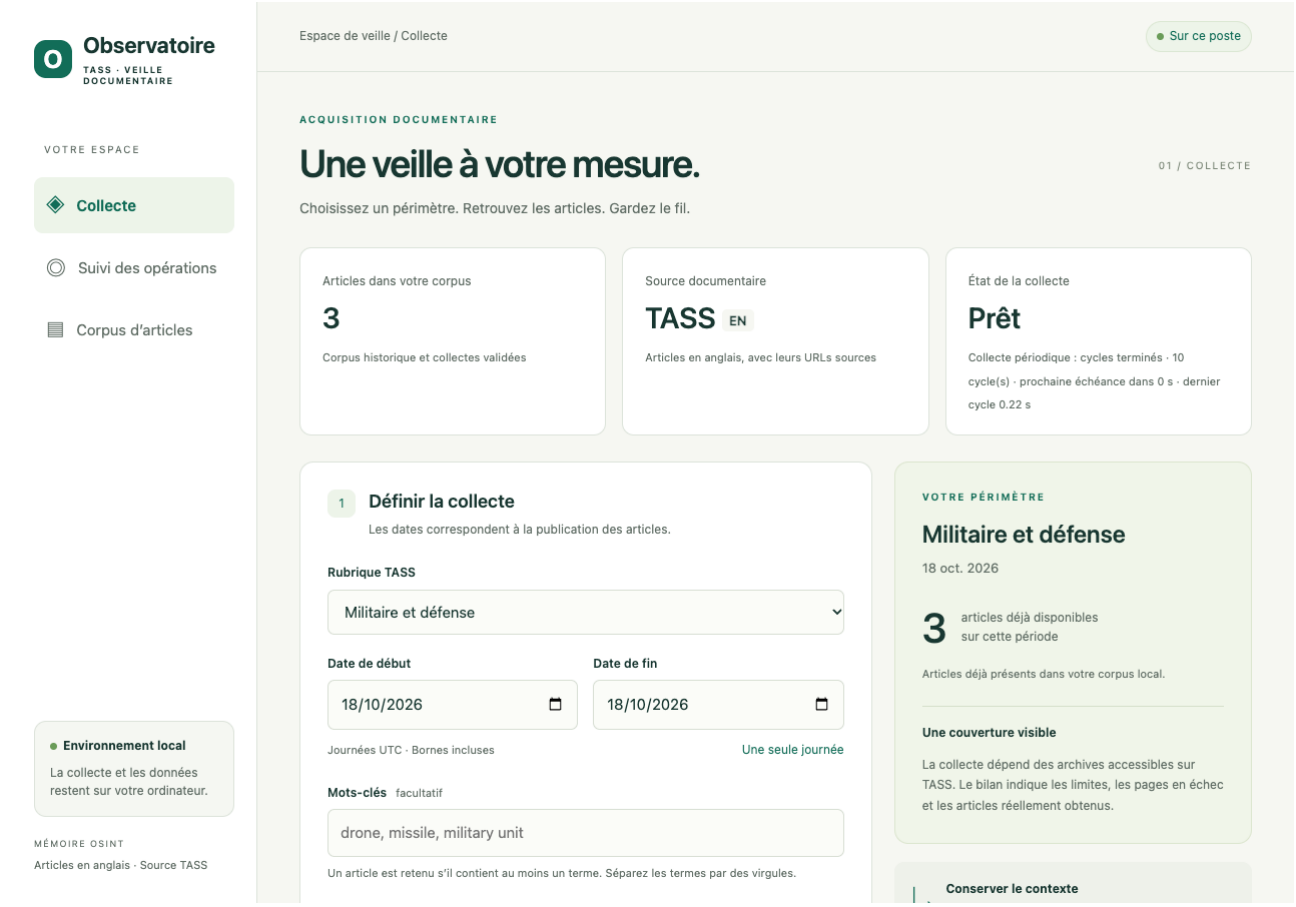


Figure 2. Collecte périodique : dix cycles sur une source fictive, sans requête TASS.

Indicateurs et extension

Actualisation chaque seconde : découvert, téléchargé, validé, nouveau, modifié, inchangé, filtré, supprimé, rejeté, durée, débit, tentatives, étape et dernière activité. Le statut indique aussi backend, fraîcheur du secours et sécurité. Les événements stage_duration ajoutent les durées par étape sans modifier le contrat des articles.

POLL_STALLED signale un worker sans activité récente. POLL_COVERAGE_LIMITED signale un quota ou une pagination limitant la couverture. Les alertes de lenteur, permissions et intégrité restent actives. Les seuils sont des choix du projet. Les alertes restent locales, sans destinataire externe.

Documentation d’utilisation

Application personnelle sur le Mac

Préparer Preparer_volume_chiffre.command dans Terminal, saisir le mot de passe, puis ouvrir Lancer_Observatoire_TASS.command. Le service doit être arrêté pendant migration et restauration. Le lanceur garde un pointeur privé vers l’état chiffré.

```
python3 -m pipeline collect-ui --state /Volumes/MemoireOSINT/B3 \
  --require-encrypted --watch-config config/collecte_continue.json
python3 -m pipeline collect-check-security \
  --state /Volumes/MemoireOSINT/B3 --require-encrypted
python3 -m pipeline collect-restore --state /Volumes/MemoireOSINT/B3
```

Collecte, consultation et droits

Choisir rubrique, période UTC, mots-clés OU et limite. La limite concerne les candidats vérifiés, même déjà connus. Le catalogue publie automatiquement après qualité ; Importer dans Bloc 2 est une action explicite. Les fichiers JSON/JSONL téléchargés en dehors du volume redeviennent des copies sous responsabilité de leur destinataire.

```
python3 -m pipeline collect-rectify ID --request-ref DOSSIER-001 \
  --changes /Volumes/MemoireOSINT/correction.json \
  --state /Volumes/MemoireOSINT/B3 --propagate-b2
python3 -m pipeline collect-erase ID --request-ref DOSSIER-002 \
  --state /Volumes/MemoireOSINT/B3 --propagate-b2
```

Une erreur de propagation Bloc 2 conserve un état d’échec et peut être rejouée. Le mode de secours bloque les modifications ; restaurer hors serveur après diagnostic. Pour reproduire sur Linux, utiliser exclusivement les fixtures et suivre Collecte_TASS.md ; l’outil de volume est propre à macOS.

Conclusion, soutenance et sources

Résultat et limites

La collecte périodique, la qualité, les versions, la reprise et le suivi sont implémentés et testés sur des arrivées fictives. La cadence est configurable. Les droits, la lecture de secours et la restauration gardent leurs preuves distinctes. Le volume chiffré conserve son périmètre documenté.

La machine demeure un domaine unique de panne. Le miroir maintient la lecture mais ne constitue pas une haute disponibilité complète. Les archives TASS peuvent être partielles. Un quota saturé demande une revue de la fenêtre et de la pagination, sans promesse d'exhaustivité. L'import Bloc 2 conserve une commande explicite et Bloc 4 requiert une inférence sur le texte identifié, sans réentraînement automatique.

Temps	Preuve à expliquer
0:00 à 0:45	Besoin, source TASS et bornes UTC.
0:45 à 1:45	Diagramme, contrat du texte et séparation B2/B4.
1:45 à 3:00	Qualité, logs et reprise au checkpoint.
3:00 à 4:20	Monitoring, benchmark et lecture de secours.
4:20 à 5:00	Droits, protection et limites observées.

Les huit critères d'oral restent prévus jusqu'à une répétition réelle. Questions : comment éviter la réintroduction d'un texte corrigé ? Que continue-t-on lors d'une panne SQLite ? Pourquoi un nouveau corpus ne demande-t-il pas un nouvel entraînement ?

Sources : grille RNCP38777, Bloc 3!A6:A38 ; complément de consigne du Bloc 3 reçu le 18 octobre 2026 ; code et preuves de cette version. Dépôt : https://github.com/edcdataworker/memoire-osint. Les documents descriptifs ne remplacent pas les tests OSINT.